Intensive Training Plan: Production React Native + Expo in 20 Study Days

20 study days inside 30 calendar days. At the end, a trainee can join a production React Native + Expo team, take a ticket with a design, build the screen the way this codebase expects, and open a clean pull request (PR).

This is **not** a basics course. Foundations get two fast refresher days. After that, every day is a medium-difficulty, real-world task built **inside this repo**: screens from a design spec, data features, and reviews like a real team.

How the 30 days are laid out

Week	Calendar days	Study days	Theme	Weekend deliverable
1	1–7	Days 1–5	Fast foundations, React Native UI, routing	PR: product detail redesign
2	8–14	Days 6–10	The data layer: API, zod, React Query, forms	PR: browse & filter, stock, form v2
3	15–21	Days 11–15	Architecture: a new module, auth, testing	PR: `users` + `auth` modules
4	22–28	Days 16–20	Production hardening, performance, shipping, capstone	PR: `carts` module (capstone)
–	29–30	–	Capstone demo, review fixes, readiness check	Final sign-off

Each week: **Monday–Friday = study days** (4–5 h). **Saturday = review day**: finish the week's PR, answer review comments, catch up. **Sunday = rest.**

Daily rhythm (4–5 hours)

Block	Time	What
Concepts	45–60 min	Read the day's links. Skim; come back when you need details.
Build	2.5–3 h	The tasks, on a branch: `git checkout -b day-07-prefetch`.
Ship	30–45 min	`npm run validate`, Conventional Commit, push, open a PR, self-review.
Journal	10 min	3 things learned, 1 thing still unclear, 1 question for the mentor.

Rules of the training

1. **Every task ends in a PR** against your fork, with screenshots (light **and** dark mode).
2. `npm run validate` **must pass** before you ask for review.
3. Read ARCHITECTURE.md, the module reference and the feature reference before Day 3.
4. Stuck for 30 minutes? Ask, with: what you tried, the exact error, a link to your branch.
5. "Stretch" tasks are optional: only after the main tasks are reviewed.

Task tags: **[M]** medium (the normal level here) · **[M+]** medium-hard (agree on the approach with the mentor first) · **[Screen]** a screen built from a design spec (see below).

How screen tasks work (real-world UI tickets)

In a real team you get a **ticket + a design**, and you build it with the team's components and tokens. Screen tasks in this plan work the same way.

1. The design source: the shadcn/ui Figma kit

This app's UI primitives are React Native Reusables, a React Native port of **shadcn/ui**. So the matching design source is a shadcn/ui Figma kit: same components, same token names (`primary`, `muted`, `card`, `border`…).

Kit	Link	Use it for
Obra shadcn/ui, Community Edition (MIT)	Figma Community · site	**Main kit.** Compose every screen in this plan
shadcn/ui kits listed by shadcn	ui.shadcn.com/docs/figma	Other free kits with the same components
Material 3 Design Kit (Google)	Figma Community · m3.material.io	Android patterns: navigation, sheets, spacing
Apple Design Resources (iOS kits)	developer.apple.com/design/resources · Human Interface Guidelines	iOS patterns: tab bars, modals, lists

Setup (once): create a free Figma account, open the Obra kit, click **"Open in Figma" / duplicate**. You now have your own copy with every component.

2. The workflow for every [Screen] task

1. **Read the screen spec** in the task: purpose, wireframe, components, tokens, states.
2. **Compose it in Figma (30 min max)** in your copy of the Obra kit: drag in Card, Button, Badge, Input… Make a light and a dark version. This is your "design". Attach a screenshot to the PR. (A mentor may give you their own Figma frame instead.)

3. **Map every Figma component to the repo** with the table below. If one is missing, add it with the React Native Reusables CLI (`npx @react-native-reusables/cli@latest add <name> --styling-library nativewind`), never by hand-writing a new primitive.
4. **Copy the closest existing pattern** (table below) instead of starting from an empty file.
5. **Build all states:** loading (skeleton), error (retry), empty, data, and dark mode.
6. **Compare** your screen with the Figma frame side by side before opening the PR.

3. Figma component → repo component

In the Figma kit	In the repo
Button (default, secondary, outline, ghost, destructive, link)	`Button` from `@/shared/ui/button` (`variant=…`, `size=…`)
Typography (h1–h4, large, muted, small, lead)	`Text` from `@/shared/ui/text` (`variant="h3"`, `"muted"` …)
Card	`Card` (`@/shared/ui/card`) or `rounded-lg border border-border bg-card p-4`
Badge	`Badge` (`@/shared/ui/badge`)
Input, Textarea, Label	`Input`, `Textarea` + `FormField` (`@/shared/components/form-field`)
Checkbox, Switch, Radio, Toggle group	`@/shared/ui/checkbox` · `switch` · `radio-group` · `toggle-group`
Tabs	`@/shared/ui/tabs`
Dialog, Alert dialog	`@/shared/ui/dialog` · `@/shared/ui/alert-dialog`
Drawer / Sheet (bottom)	`AppBottomSheet` (`@/shared/components/app-bottom-sheet`)
Avatar	`@/shared/ui/avatar`
Skeleton	`@/shared/ui/skeleton`
Alert	`@/shared/ui/alert`
Separator, Progress	`@/shared/ui/separator` · `@/shared/ui/progress`
Icons (Lucide)	`Icon` (`@/shared/ui/icon`) with an icon from `lucide-react-native`
Empty / error / loading blocks	`EmptyState`, `ErrorState`, `LoadingState` (`@/shared/components/screen-states`)
Settings list rows	`SettingsGroup`, `SettingsRow` (`@/shared/components/settings-row`)

Open **Settings** → **UI Sandbox** in the app to see every one of these live, in both themes.

4. Theme tokens (never hex colors)

Use for	Classes
Screen background / main text	`bg-background` · `text-foreground`
Cards, sheets	`bg-card` · `border-border` · `rounded-lg`
Secondary text, captions	`text-muted-foreground` (or `<Text variant="muted">`)
Subtle surfaces, chips	`bg-muted` · `bg-secondary` / `text-secondary-foreground`
Brand actions, prices, links	`bg-primary` / `text-primary-foreground` · `text-primary`
Tinted highlight	`bg-primary/10` (any token + `/opacity`)
Errors, destructive actions	`bg-destructive` · `text-destructive`
Success, "in stock", discounts	`bg-success` / `text-success-foreground`
Inputs and focus	`border-input` · `ring-ring`

Tokens are defined in `src/global.css` (light + dark). For places that need a color **value** instead of a class (tab bar, `ActivityIndicator`), use `useThemeColors()`. Details: THEMING.md.

5. Existing patterns to copy

You need…	Copy from
A list screen with search, infinite scroll, pull to refresh	`catalog/features/product-list/product-list-screen.tsx`
A list row / card	`catalog/features/product-list/product-card.tsx`
A skeleton for a list	`catalog/features/product-list/product-list-skeleton.tsx`
A detail screen with a fixed bottom action bar	`catalog/features/product-detail/product-detail-screen.tsx`
A form with validation and a submit error	`catalog/components/product-form.tsx`
A bottom sheet with a list of choices	`catalog/components/category-picker-sheet.tsx`
An action sheet (View / Edit / Delete)	`catalog/features/product-list/product-actions-sheet.tsx`
A confirm dialog	`catalog/features/delete-product/delete-product-dialog.tsx`
A horizontal carousel widget	`catalog/features/top-rated/top-rated-carousel.tsx`
A dashboard with stat cards	`home/features/dashboard/home-screen.tsx`
A settings-style screen	`settings/features/app-settings/settings-screen.tsx`

You need…	Copy from
A floating "+" button	`FloatingActionButton` (`@/shared/components/floating-action-button`)

Spacing rhythm used in the app: screen padding `p-4`, gaps `gap-3` / `gap-4` / `gap-6` between sections, cards `rounded-lg border border-border bg-card p-4`. Bottom bars add `useSafeAreaInsets().bottom`.

Before Day 1: entry check (1–2 hours)

Each trainee must answer **yes** to all of these. If not, fill the gap during the first weekend.

- [ ] I write `async` / `await` with `try/catch` and can explain a Promise. (javascript.info: Promises, async/await)
- [ ] I use `map`, `filter`, `find`, `reduce`, destructuring and spread fluently. (javascript.info)
- [ ] I can type functions, objects and unions in TypeScript. (TypeScript handbook)
- [ ] I can build a React component with props and state and render a list with keys. (react.dev/learn)
- [ ] I can branch, commit, push and open a pull request. (Pro Git book)

Setup: Node.js LTS (20.19.4+), VS Code (ESLint, Prettier, Tailwind CSS IntelliSense), Git, GitHub, a free Figma account, **Expo Go for SDK 58** on a phone. Fork this repo, `npm install`, `npm start`, open it in Expo Go (GETTING-STARTED.md).

Week 1: Fast foundations, React Native UI and routing

Day 1 — JavaScript + TypeScript power refresher

Goal: Write the typed, async utility code production apps are full of.

Concepts (fast): `Promise.all` vs `allSettled`, `AbortController`, closures, immutability with spread. TypeScript narrowing, **discriminated unions**, exhaustive `switch` with `never`, generics with constraints, `Partial` / `Pick` / `Omit` / `Record`, `as const`, `satisfies`, `unknown` vs `any`.

Read:

- TypeScript: Narrowing · Generics · Utility types

Tasks:

1. **[M] Typed retry helper.** `src/shared/lib/retry.ts`: `retry<T>(fn: (attempt: number) => Promise<T>, options: { retries: number; delayMs: number; signal?: AbortSignal }): Promise<T>` with exponential backoff (`delayMs * 2 ** attempt`), stopping immediately on abort. Tests in `src/shared/lib/__tests__/retry.test.ts` with Jest fake timers: first-try success, 3rd-try success, gives up after `retries`, stops on abort.
2. **[M] Request-state union.** Model `RequestState<T> = idle | loading | success(data) | error(ApiError)` as a discriminated union and write `describeState(state)` with an exhaustive `switch`. Adding a status must break the build.
3. **[M] Read the error model.** Read `src/shared/api/http-client.ts` and `api-error.ts`. In the PR, explain when each `ApiError.kind` is thrown and why a cancelled request is **not** an `ApiError`.

Done when:

- [ ] `retry` has 4+ tests, no `any`. Removing a `switch` case fails `npm run typecheck`.

Day 2 — React fast-track (the parts juniors get wrong)

Goal: Use state, effects and custom hooks the way experienced React developers do.

Concepts (fast): state as a snapshot, derived values, effects only for external systems, cleanup, custom hooks, composition with `children`. This repo has the **React Compiler** on, so you rarely need `useMemo` / `useCallback`.

Read:

- React: You Might Not Need an Effect
- React TypeScript cheatsheet · Expo: React Compiler

Tasks:

1. **[M] `<Rating />` component.** `src/modules/catalog/components/rating.tsx`: `<Rating value={4.6} max={5} size="sm" | "md" showValue />` with full, half and empty stars (lucide `Star` / `StarHalf`) and an accessibility label "Rated 4.6 out of 5". Replace the hand-made stars in `product-card.tsx`, `product-detail-screen.tsx` and `top-rated-carousel.tsx`. Add it to `CatalogShowcase` (UI Sandbox → Modules).
2. **[M] Re-implement from memory.** Write `useDebouncedValue(value, ms)` without looking, then compare with `src/shared/hooks/use-debounced-value.ts` and explain the differences in the PR.
3. **[M] `useInterval` + `useToggle`** in `src/shared/hooks/`. `useInterval(cb, delayMs | null)` must always call the **latest** callback and stop on `null`. Demo: a pausable counter in a "Hooks" section of the UI Sandbox (Shared tab).

Done when:

- [ ] Three screens use `<Rating />` and look right in both themes. The interval doesn't leak after navigating away.

Day 3 — Screen build: product detail redesign (part 1)

Goal: Turn a spec into a polished native screen using the team's components and tokens.

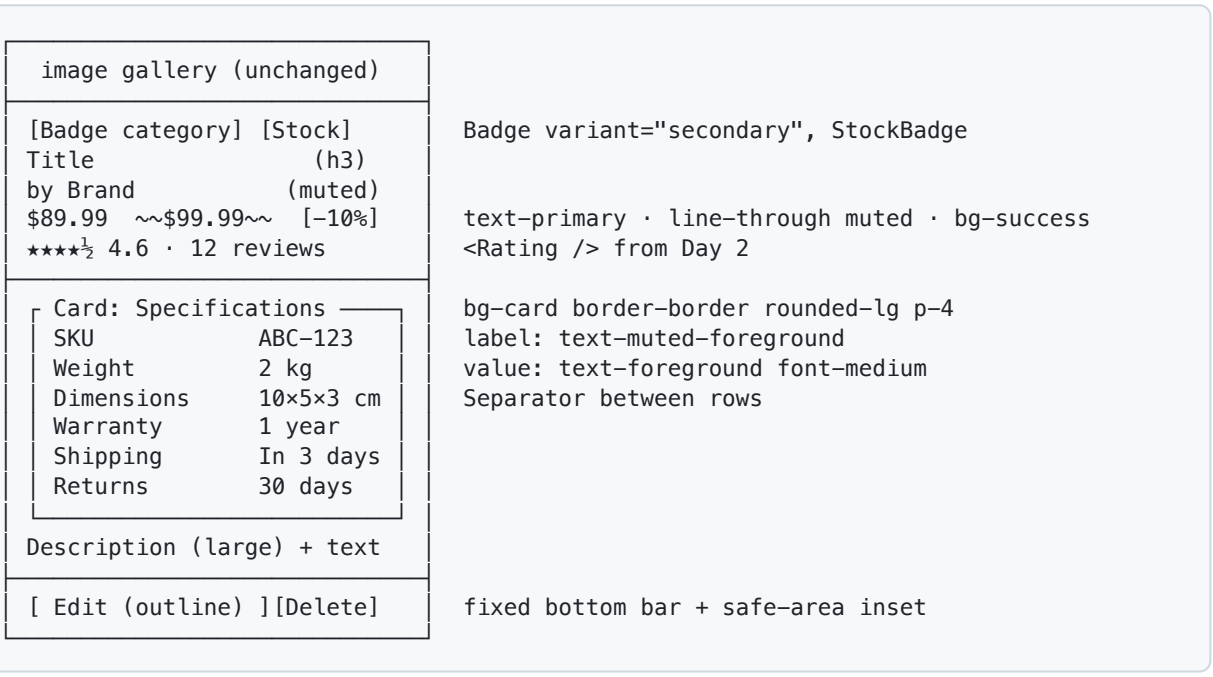
Concepts (fast): Flexbox in React Native (column by default), NativeWind + tokens, dark mode, safe areas, `useWindowDimensions`. The codebase map: `src/app` → `src/modules` → `src/shared`.

Read:

- ARCHITECTURE.md and THEMING.md (required)
- React Native: Flexbox · NativeWind docs · Expo: Safe areas

Tasks:

1. **[Screen][M] Product detail v2.** Compose it in Figma first (see "How screen tasks work").



   Data: add `sku`, `weight`, `dimensions`, `warrantyInformation`, `shippingInformation`, `availabilityStatus`, `returnPolicy`, `minimumOrderQuantity` to `productSchema` (optional or with defaults, so created products still parse; `curl https://dummyjson.com/products/1` to see them). Hide rows with no value. Pattern to copy: the existing `product-detail-screen.tsx` layout.

2. **[M] Tablet layout.** At ≥ 768 px wide: product list in **2 columns** (FlashList `numColumns`); detail screen as image left / info right. Phones unchanged.

3. **[M] Trace one request.** Diagram from tapping the Products tab to `fetch` and back, naming every file, in the PR description.

Done when:

- [ ] Figma frame + app screenshots (phone light/dark, tablet) in the PR. Schema tests pass.

Day 4 — Expo Router in depth + reviews screen

Goal: Design navigation like a production app: stacks, tabs, modals, typed params, deep links.

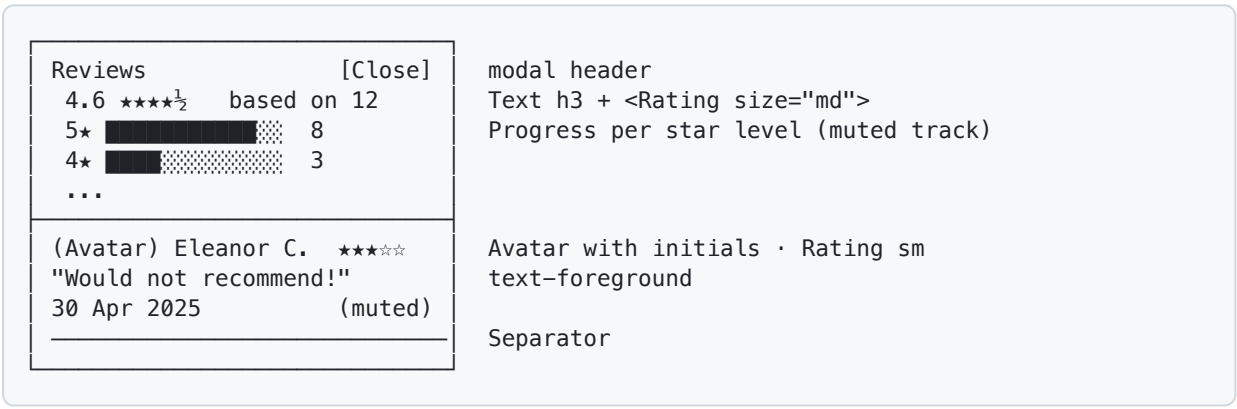
Concepts (fast): layouts, `Stack` / `Tabs`, groups, dynamic routes, thin route files that pass params as props, typed routes and route builders, modals, search params, `router.setParams`, deep links.

Read:

- Expo Router: introduction · URL parameters · Modals · Typed routes

Tasks:

1. **[Screen][M] Reviews modal.** New feature `catalog/features/product-reviews/` (`index.ts`, `README.md`). Route `/products/[id]/reviews`, presented as a **modal**, route builder `catalogRoutes.reviews(id)`, opened from a "See all N reviews" link on the detail screen. Add `reviewSchema` (`rating`, `comment`, `date`, `reviewerName`) to the product schema.



   Pattern to copy: the list structure from `product-list-screen.tsx`.

2. **[M] Search in the URL.** The product list reads its search from `?q=` and updates it with `router.setParams`, so `/products?q=phone` opens pre-filtered and survives navigation.

3. **[M] Deep links.** Open `/products/5` and `/products?q=phone` directly with an Expo Go deep link (`exp://<your-ip>:8081/--/products/5`). "Back" must land on the tabs. Document the commands in the PR.

Done when:

- [ ] The modal closes with swipe down (iOS) and back (Android). No hand-typed paths.

Day 5 — Lists, media and interaction quality

Goal: Lists and images that feel fast and native.

Concepts (fast): FlashList v2 (`keyExtractor` , `getItemType` , `numColumns` , headers, empty state), `expo-image` caching and transitions, why `useValueWhileFocused` exists (see ARCHITECTURE.md), haptics.

Read:

- FlashList documentation · React Native: performance

Tasks:

1. **[M] Review filters.** In the reviews modal: FlashList, newest first, filter chips "All · 5★ · 4★ · 3★ and below" (`ToggleGroup` or chips with `bg-muted` / selected `bg-primary`). Each filter has its own `EmptyState` .
2. **[Screen][M+] Full-screen gallery.** Replace the detail image `ScrollView` with a paging list and **page dots** (`bg-primary` active, `bg-muted` inactive). Tapping an image opens a modal route `/products/[id]/gallery?index=2` : black background (the one allowed exception: use `bg-black` for media viewers), close button top-right with safe-area inset, swipe between images, "2 / 5" counter.
3. **[M] Polish pass.** `haptics.light()` on chips and image taps; every tappable element has `accessibilityRole` and a label.

Stretch: pinch-to-zoom in the gallery (gesture handler + Reanimated are installed).

Done when:

- [ ] **Saturday:** Week 1 PR merged after review (Rating, detail v2, reviews, gallery, URL search).

Week 2: The data layer

Day 6 — HTTP client, errors and robust schemas

Goal: A trustworthy API layer: clear errors, safe parsing, real tests.

Concepts (fast): one HTTP client and one error type; zod `.default()`, `.optional()`, `.catch()`, `z.enum`, transforms, `z.input` vs `z.output`; `EXPO_PUBLIC_*` values are public.

Read: Zod · Expo: environment variables · Jest

Tasks:

1. **[M] Error-path tests** in `products.service.test.ts`: 404 → `kind: 'http'` + `isNotFound`; network failure → `network`; wrong JSON shape → `parse`; timeout → `timeout` (fake timers + a `fetch` that never resolves).
2. **[M] Defensive schema.** `availabilityStatus` as `z.enum(['In Stock', 'Low Stock', 'Out of Stock']).catch('In Stock')`; review `date` becomes a `Date`. Tests for both, including bad values.
3. **[M] Dev request log.** In `__DEV__` only, log `METHOD path → status (123 ms)` per request.

Stretch: an `x-request-id` header per request, included in `ApiError`.

Done when:

- [ ] 6+ new tests; `npm run validate` green.

Day 7 — React Query reads + "browse by category" screen

Goal: Fetch, cache, prefetch and compose server data.

Concepts (fast): key hierarchies and factories, `queryOptions`, `staleTime` vs `gcTime`, `select`, `enabled`, `initialData` vs `placeholderData`, prefetching, dependent queries.

Read:

- TkDodo: Practical React Query · Effective Query Keys · Query Options API
- TanStack: Prefetching · Dependent queries

Tasks:

1. **[Screen][M] Products by category.** Follow ADDING-A-FEATURE.md: endpoint, key, service + test, query, feature folder, route `/products/category/[slug]`.

   ```
   Product list (top):
   ┌──────────────────────────────────┐
   │ (⌕) Search products              │
   │ (Beauty)(Phones)(Laptops)… →     │  horizontal chips: bg-muted rounded-full px-3 py-1.5
   ├──────────────────────────────────┤
   Category screen:
   │ Smartphones          (h3)        │
   │ 16 products        (muted)       │
   │ [ProductCard] …                  │  reuse ProductCard via ../product-list
   └──────────────────────────────────┘
   ```

2. **[M] Prefetch on press-in** of a `ProductCard` (`productQueries.detail(id)`). Show the effect with the Day 6 request log.

3. **[M] "More from this category"** on the detail screen: up to 6 products from the same category, excluding the current one, as a **dependent query** (disabled until the product loads). Pattern to copy: `top-rated-carousel.tsx`.

Done when:

- [ ] No hand-written query keys; the dependent query is disabled, not just empty, at first.

Day 8 — Infinite lists, sort & filter sheet, cache consistency

Goal: Paginated data with filters, without breaking the cache.

Concepts (fast): `useInfiniteQuery`, every input in the key, cache helpers must know every list variant.

Read: TanStack Query overview and every line of `catalog/data/product.queries.ts` and `product.cache.ts`.

Tasks:

1. **[Screen][M+] Sort & filter sheet.**

   ```
   [ Sort ⇅ ] button in the list header (Button variant="outline" size="sm")
   ┌ AppBottomSheet ──────────────────┐
   │ Sort by              (large)     │
   │ ◉ Relevance                      │  RadioGroup
   │ ○ Price: low to high             │
   │ ○ Price: high to low             │
   │ ○ Best rated                     │
   │ ○ Title A–Z                      │
   │ [ Apply (primary, full) ]        │  + safe-area bottom padding
   └──────────────────────────────────┘
   ```

   `productsService.list` already supports `sortBy` and `order`. Put the sort in `productKeys.list(...)` and in the URL (`?sort=price-asc`). Pattern to copy: `category-picker-sheet.tsx`.

2. **[M+] Fix the cache helpers.** `prependProductToList` only updates `list({ search: '' })`. Decide what a new product should do in sorted lists (prepend? invalidate?), implement it, explain it in the PR. Edit and delete must still update every list.

3. **[M] Scroll-to-top** button after ~2 pages.

Done when:

- [ ] Going back to a previous sort shows cached results instantly. CRUD still updates the visible list.

Day 9 — Mutations: optimistic updates and failure UX

Goal: Change server data safely and give honest feedback.

Read:

- TkDodo: Mastering Mutations · React Query Error Handling
- TanStack: Optimistic Updates

Tasks:

1. **[Screen][M+] Stock stepper.** New feature `catalog/features/adjust-stock/`. On the detail screen: `[ − ] 24 in stock [ + ]` (outline icon buttons, number `text-xl font-bold`). Optimistic update in the detail cache and every list, `PUT /products/:id`, rollback on error. Rapid taps send **one** request with the final value (~600 ms debounce).
2. **[M] "Fail next write" dev switch** in Settings → Developer (`SettingsRow` + `Switch`): the next POST/PUT/DELETE fails with a 500 `ApiError`. Implement it in `src/shared/api`, active only in `__DEV__`.
3. **[Screen][M] Toasts.** Shared `Toast` + `useToast()` (context, auto-hide, success / error variants): `bg-card border-border` with an icon in `text-success` or `text-destructive`, slides in from the bottom above the safe area. Replace `Alert.alert` for delete and stock errors. Add it to the UI Sandbox.

Done when:

- [ ] With "Fail next write" on, stock and delete roll back and show an error toast. Five quick taps on "+" send exactly one request.

Day 10 — Forms that hold up in production

Goal: Validated, friendly forms that handle server errors and unsaved changes.

Read:

- React Hook Form: get started · useFieldArray · Expo: keyboard handling
- Every line of `catalog/components/product-form.tsx`.

Tasks:

1. **[Screen][M] Product form v2.** Add `discountPercentage` (0–90) and **tags**:

   ```
   Discount (%)                    FormField + Input keyboardType="number-pad"
   [ 10                          ]
   Tags  (up to 5)                 FormField label + hint "Press enter to add"
   [ type a tag…                 ]
   (wireless ×) (audio ×)          Badge variant="secondary" + X icon, tap to remove
   ```

   Lowercase, unique, max 5. Extend `productFormSchema` and its tests.

2. **[M] Server error on the form.** A 400 with a message shows in the form's `Alert` (`setError('root', …)`) and keeps the input. Test with the Day 9 dev switch.

3. **[M+] Unsaved-changes guard.** Leaving a dirty form asks "Discard changes?" (`AlertDialog`). Research how Expo Router / React Navigation lets you prevent leaving a screen; explain your choice.

Done when:

- [ ] **Saturday:** Week 2 PR merged (categories, sort, prefetch, stock, toasts, form v2).

Week 3: Architecture, a new module, auth and testing

Day 11 — New module: `users` (data layer)

Goal: Build a module from scratch following the architecture, without hand-holding.

Read: ADDING-A-MODULE.md (follow it) · Vertical Slice Architecture · Modular Monolith, a Primer · DummyJSON: users

Tasks:

1. **[M] `users` data layer:** endpoints, nested schema (address, company), service (`list` , `search` , `getById`), keys, queries (`useUserList` infinite + search, `useUser`), `index.ts` , `README.md` .
2. **[M] Tests:** 5+ service/schema tests including a parse failure and a 404.
3. **[M] Break each ESLint boundary rule once** (deep import from `src/app` , `src/shared` importing a module, a screen importing the service). Paste the errors into the PR, then undo.

Done when:

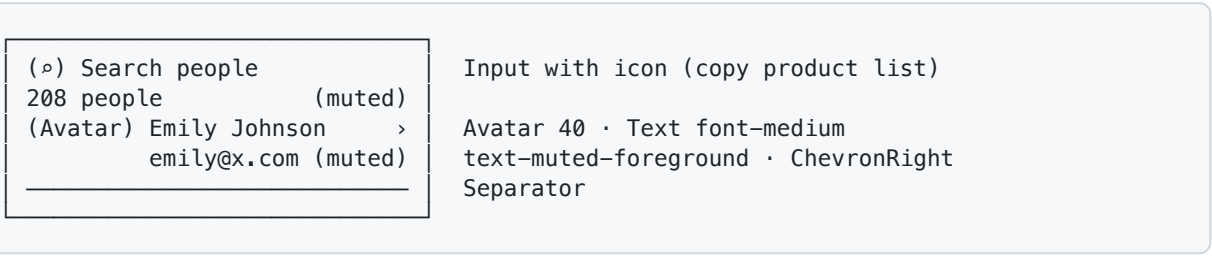
- [ ] `npm run validate` green; the module README matches the reference's structure.

Day 12 — Screen build: Users list, user profile and a Home widget

Goal: Ship screens on your module and expose a widget through its public API.

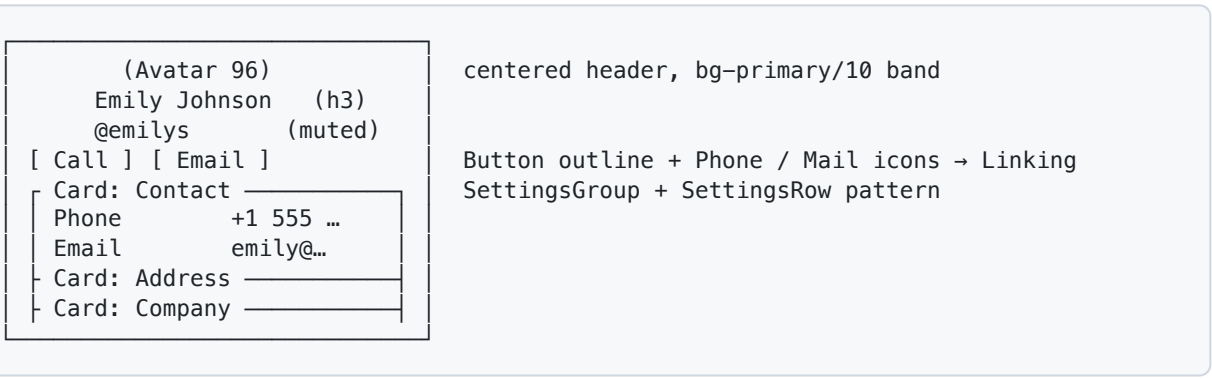
Tasks:

1. **[Screen][M] Users list** (`features/user-list` , new **Users** tab):



   Infinite scroll, debounced search, skeleton rows, empty and error states, pull to refresh.

2. **[Screen][M] User profile** (`features/user-detail` , route `/users/[id]`):



3. **[M] Home "Team" widget.** Export `UserAvatarsRow` (5 overlapping avatars + "View all") from `@/modules/users` and place it on the Home dashboard. Add a `UsersShowcase` to the UI Sandbox "Modules" tab.

Done when:

- [ ] Home imports only from `@/modules/users` . Each feature has a README.

Day 13 — Auth module: login screen and secure tokens

Goal: Real login, tokens stored securely and sent on every request.

Concepts (fast): access vs refresh tokens; never plain storage for tokens; dependency inversion: `src/shared/api` can't import `auth`, so `auth` **registers** a token getter.

Read: DummyJSON: auth (user `emilys` / `emilyspass`) · Expo SecureStore · Expo: storing data

Tasks:

1. **[Screen][M] Login screen** (`src/modules/auth/features/login`):



   react-hook-form + zod; keyboard-aware; `npx expo install expo-secure-store` (works in Expo Go). Pattern to copy: `product-form.tsx`.

2. **[M+] Token injection.** `setAuthTokenGetter(fn)` in `src/shared/api`; `http-client.ts` adds `Authorization: Bearer …` when registered. Test that the header is sent.

3. **[M] Session.** `useSession()` (`GET /auth/me`) and `logout()` (clears SecureStore and the query cache). Settings shows an "Account" group: avatar, name, "Log out" (`destructive`).

Done when:

- [ ] Reopen the app: still signed in. Log out: tokens gone. `src/shared` imports no module.

Day 14 — Protected routes and token refresh

Goal: Gate the app behind login and survive expired tokens.

Read: Expo Router: authentication · Protected routes

Tasks:

1. **[M+] Protected routes.** Signed-out users only see `/login`; signed-in users never see it. Keep the splash screen until the session is known (no flash of the wrong screen).
2. **[M+] Refresh on 401.** On 401: `POST /auth/refresh` **once**, store new tokens, retry. Several 401s at the same time trigger only **one** refresh. Refresh fails → log out. Test the "many 401s, one refresh" case.
3. **[M] Redirect after login** to the deep link the user originally opened.

Stretch: log in with `expiresInMins: 1` and prove the app keeps working after 2 minutes.

Day 15 — Testing that catches real bugs

Goal: Test components and hooks, not just functions.

Read: Expo: unit testing · React Native Testing Library · TanStack: testing · TkDodo: Testing React Query

Tasks:

1. **[M] Setup:** React Native Testing Library + a `renderWithProviders()` helper (fresh `QueryClient` per test, retries off).
2. **[M] Component tests:** login (validation errors, disabled while submitting, server error); `ProductCard` (title, discounted price, `onPress`); your `Toast`.
3. **[M+] Hook test:** `useDeleteProduct` with a failing service: removed optimistically, then restored.

Done when:

- [ ] 10+ new tests, green in CI. **Saturday:** Week 3 PR merged (`users`, `auth`, tests).

Week 4: Production hardening, shipping and the capstone

Day 16 — Resilience and accessibility

Read: Expo Router: error handling · React Native: accessibility · expo-network · onlineManager

Tasks:

1. **[Screen][M] Offline banner:** a slim bar under the header, `bg-muted` + `text-muted-foreground` + `WifiOff` icon, "You're offline. Showing saved data." Driven by `onlineManager` (wired in `src/shared/lib/query-client.ts`). Test with airplane mode.
2. **[Screen][M] Error screens:** each route group shows a friendly error (icon, title, "Try again" button) instead of crashing the app. Crash a screen on purpose to prove it.
3. **[M] Accessibility audit** with VoiceOver / TalkBack on Products, Users and Login. Fix labels, roles, focus order, touch targets ≥ 44 pt. List the fixes in the PR.

Day 17 — Performance and debugging

Read: Expo: debugging tools · React Native: performance · React: memo

Tasks:

1. **[M] Profile** the product list (scroll + search) with React DevTools Profiler. Record the 3 slowest renders and their causes.
2. **[M+] Fix the worst one** (image sizes, re-renders, expensive `select`, item types). Before / after numbers in the PR.
3. **[M] Bundle check:** `npx expo export`; list the 5 biggest dependencies and whether any is avoidable.

Day 18 — Builds, environments, releases and CI

Read: Development builds · EAS Build · eas.json · EAS Update · Deployment patterns · GitHub Actions · CODEOWNERS

Tasks:

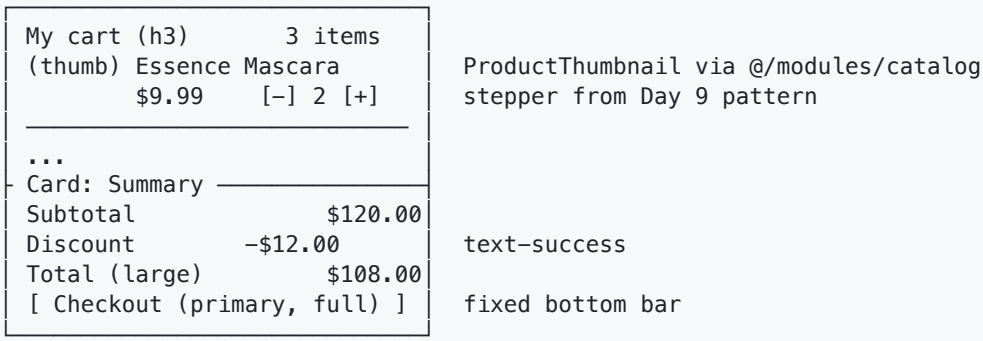
1. **[M]** `eas.json` with `development`, `preview`, `production` profiles, each with its own `EXPO_PUBLIC_API_URL` and update channel. Explain who uses which.
2. **[M] CI:** add `npx expo export --platform android` to `.github/workflows/ci.yml` so a broken bundle fails CI.
3. **[M]** `CODEOWNERS`**:** map each `src/modules/<name>/` to a (placeholder) team and `src/shared/` to "platform".

Stretch: ship a JS-only change to a preview build with `eas update`.

Day 19 — Capstone part 1: `carts` module

Build a full module alone. The mentor only reviews. Read DummyJSON: carts.

1. **[M] Data layer:** `/carts/user/:userId`, `/carts/:id`, `/carts/add`, `PUT /carts/:id`, schemas, service, keys, queries, tests, README.
2. **[Screen][M] Cart screen** (new **Cart** tab with a `Badge` item count):



   Empty state: `ShoppingCart` icon, "Your cart is empty", "Browse products" button.

Day 20 — Capstone part 2: writes, polish, presentation

1. **[M+] "Add to cart"** on the product detail screen, exported as a widget from `@/modules/carts` (catalog must not import carts internals). Quantity changes and removal with **optimistic updates** and rollback. DummyJSON doesn't save carts: handle it like the products workaround and document it.
2. **[Screen][M] Order summary modal** after "Checkout": success icon (`text-success`), order lines, total, "Back to shopping".
3. **[M]** All states (loading, error, empty, offline, signed out), accessibility, light/dark screenshots, READMEs for the module and each feature.

Done when:

- [ ] Capstone PR open, CI green, self-reviewed against the PR template.

Calendar days 29–30: Demo and readiness check

Day 29: Demo (30 min per trainee) on a real phone, then explain: one request from tap to `fetch` in `carts`; why `src/shared` can't import a module and how tokens still reach the HTTP client; what happens in the cache when "add to cart" fails.

Day 30: Retro and sign-off. Merge the capstone, write a one-page retrospective, and go through the readiness checklist with the mentor.

Production-readiness checklist

A trainee is ready when they can, **without help**:

- [ ] Turn a Figma frame into a screen using `src/shared/ui`, theme tokens and existing patterns.
- [ ] Add a module and a feature that pass ESLint boundaries and match the reference READMEs.
- [ ] Write zod schemas from a real API and tests for the service, including failures.
- [ ] Use React Query well: key factories, `queryOptions`, infinite and dependent queries, prefetching, cache updates vs invalidation, optimistic updates with rollback.
- [ ] Build validated forms with server errors and unsaved-change protection.
- [ ] Handle loading, error, empty, offline and signed-out states everywhere.
- [ ] Implement token auth with secure storage, protected routes and refresh.
- [ ] Profile and fix a performance problem with evidence.
- [ ] Support dark mode and screen readers.
- [ ] Explain development builds, EAS Build profiles, EAS Update channels and CI.
- [ ] Work in small PRs with Conventional Commits and screenshots, and handle review.

Mentor review rubric (per PR)

Area	0: missing	1: works	2: production quality
Correctness	Doesn't meet the task	Happy path works	Error, empty, offline and edge cases handled
Design match	Ignores the spec	Roughly matches	Matches the Figma frame in both themes, consistent spacing
Architecture	Wrong folders, deep imports	Right folders	Clean public APIs, nothing leaks, READMEs updated
Data layer	Fetch in UI, no validation	Hooks + service + schema	Correct keys, cache consistency, failure tests

Area	0: missing	1: works	2: production quality
UI quality	Hex colors, broken dark mode	Works in one theme	Tokens, dark mode, accessibility, haptics, no layout jumps
Code quality	`any`, dead code, logs	Typed, readable	Small functions, clear names, no duplication
Workflow	One huge commit, red CI	Green CI, description	Small commits, screenshots, reasoning explained

Target: **2 in every area** by the capstone.

After the training (learn on the job)

Push notifications, analytics and crash reporting, internationalisation, advanced animations with Reanimated, offline-first storage and sync, in-app purchases, store review and release, production monitoring.

Reference links

Topic	Links
This repo	ARCHITECTURE · ADDING-A-MODULE · ADDING-A-FEATURE · CONVENTIONS · THEMING · LIBRARIES
Design	Obra shadcn/ui kit · shadcn Figma kits · Material 3 kit · Apple Design Resources · Human Interface Guidelines
Architecture	Vertical Slice Architecture · Modular Monolith · Feature-Sliced Design · Bulletproof React
React Native/Expo	React Native · Expo tutorial · Expo Router · NativeWind · React Native Reusables
Data	TanStack Query · TkDodo's blog · Zod · React Hook Form
Quality	ESLint no-restricted-imports · Jest · Testing Library · Conventional Commits